SENTIMENT CLASSIFICATION FOR NIGERIAN PIDGIN

SENTIMENT CLASSIFICATION FOR NIGERIAN PIDGIN, AND WHETHER IT TRANSFERS TO FINANCIAL DISCUSSION

Author: HonourGod Kilechukwu Levison **Course:** Machine Learning Techniques I - Summative Project **Date:** October 2026

Project Links: **GitHub Repository:** GitHub **Live Application:** Live Application **Fine-tuned Model:** Fine-tuned Model **Demo Video:** Demo Video

ABSTRACT

This project builds a three-way sentiment classifier for Nigerian Pidgin and then tests whether it survives a change of subject matter. The training data is the Nigerian Pidgin portion of AfriSenti, the corpus used for SemEval-2023 Task 12. I fine-tuned AfroXLM-R and compared it against a TF-IDF baseline, a Word2Vec and BiLSTM model, and three other pretrained transformers.

Two problems in the released benchmark shaped the work. The neutral class is 1.5% of the training split and 10.4% of the test split, and 1,013 training tweets appear word for word in the test set, with 124 of them carrying different labels in the two splits. I removed the overlap and trained on 3,486 rows instead of the released 5,121.

The final model reaches 0.697 accuracy on the test split, averaged over three seeds, against the 0.676 reported for the same model and language by Muhammad et al. (2023b). It scores an F1 of exactly 0.000 on the neutral class on every seed, having never once predicted that class across 431 neutral test examples.

Because no labelled Pidgin data exists for financial text, I collected and hand-labelled 93 Pidgin posts about money, crypto and peer-to-peer trading. Measured against a length-matched sample of the AfriSenti test set, accuracy falls from 0.759 to 0.355. A second annotator labelled the set independently; three-way agreement was low at Cohen's kappa of 0.088, concentrated on the neutral class, so the 73 disagreements were adjudicated jointly to produce the reported set of 93 posts. The result is unchanged by that adjudication. That drop separates into 24.6 points caused by the model's inability to predict neutral meeting a domain that is 44.1% neutral, and 15.8 points of genuine domain difficulty. The model carries its training prior across unchanged: it predicts negative 60.6% of the time on financial posts where only 16.5% are negative.

The conclusion is that a model can look competitive on a benchmark and still be unusable in the setting it would be deployed in, and that the cause here is the composition of the only labelled data that exists for this language.

1 INTRODUCTION

Nigerian Pidgin, written pcm in ISO 639-3, is an English-lexifier creole spoken across Nigeria and neighbouring countries. Ethnologue's 2025 figures put it at roughly 121 million speakers, of whom about 5 million use it as a first language and 116 million as a second, which makes it the most widely spoken language in Africa and around the fourteenth most spoken in the world. Against a Nigerian population of roughly 240 million, that is close to half the country. Adelani et al. (2024) cite a comparable figure of about 120 million. It has no official status and is used almost entirely informally, which is part of why it is under-resourced: the written Pidgin that exists is mostly social media and forum text rather than the news and government corpora that NLP usually draws on.

The second half of the problem is where that language meets money. Nigeria ranks third in the Chainalysis Global Crypto Adoption Index for 2026 and, more to the point here, first in the world for domestic peer-to-peer cryptocurrency activity. Earlier editions placed it second in 2024 and sixth in 2025, so the exact rank moves, but the pattern does not: a very large volume of ordinary retail crypto activity, much of it peer-to-peer, conducted by a population half of whom speak Pidgin.

Those two facts meet on Nigerian forums and social media, and they create a practical gap. A company running an exchange or a payment service in Nigeria, and anyone studying how users talk about one, needs to read sentiment in Pidgin. If they do not speak it themselves, they need a model. This project asks whether the models that exist are fit for that.

I chose this language because I speak it, which matters for a project that requires reading model errors and judging whether a prediction is right. Error analysis on a language you cannot read reduces to repeating whatever the metric says. I also follow Nigerian crypto discussion closely, which helped when deciding whether a post expressed a stance or was simply explaining how something works.

Sentiment models for Pidgin already exist. What does not exist, as far as I could find, is any labelled Pidgin data for the financial domain, so nobody has measured whether those models transfer to it. That gap is what this project addresses.

The objective is to train a Pidgin sentiment classifier, evaluate it properly against published results, and then measure what happens when the subject matter changes. Two research questions follow:

1. Which kind of pretraining works best for an English-lexifier creole: English-only, general multilingual, or African-language-adapted?
2. Does a model trained on general Pidgin social media text still work on Pidgin financial discussion, and if not, why not?

2 RELATED WORK

Work on Nigerian Pidgin sentiment runs through two linked corpora. NaijaSenti (Muhammad et al., 2022) is a Nigerian Twitter sentiment corpus covering Hausa, Igbo, Nigerian Pidgin and Yoruba. AfriSenti (Muhammad et al., 2023b) extends the same effort to 14 African languages and supplied the data for SemEval-2023 Task 12 (Muhammad et al., 2023a). Both are three-way sentiment tasks with positive, negative and neutral labels.

The two releases overlap more than their separate names suggest. I downloaded the Nigerian Pidgin files from both repositories and compared them: the train, dev and test files are byte-for-byte identical, with matching SHA-256 hashes. For Pidgin, they are the same data.

On the modelling side, the relevant comparison is between kinds of pretraining. XLM-R (Conneau et al., 2020) is trained on 100 languages, which do not include Nigerian Pidgin. AfroXLM-R (Alabi et al., 2022) adapts XLM-R to 17 African languages, including Pidgin, through multilingual adaptive fine-tuning, removing non-African-script vocabulary from the embedding layer in the process. AfriBERTa (Ogueji et al., 2021) takes the opposite approach, pretraining from scratch on 11 African languages using under 1 GB of mostly news text.

Muhammad et al. (2023b) report baselines for all three on the Pidgin split in their Table 7: AfriBERTa-large 62.3, XLM-R-base 63.8 and AfroXLMR-base 67.6 accuracy, averaged over five runs. They note in their discussion that AfroXLMR-base performs slightly better on Pidgin than the other models of comparable size.

Two gaps motivated this project. First, I found no published audit of the AfriSenti Pidgin release itself, and the audit in Section 4 turns up problems that affect any result computed on it. Second, I found no labelled Pidgin sentiment data for financial text, which means the transfer question in Section 7 has not previously been answerable.

3 DATASET AND DATA PREPARATION

3.1 Source and licence

The corpus is the Nigerian Pidgin portion of AfriSenti, downloaded from the official SemEval-2023 Task 12 repository rather than from a mirror. The released splits are 5,121 training rows, 1,281 dev and 4,154 test, which match the sizes reported in Table 6 of Muhammad et al. (2023b) and confirm these are the official splits.

AfriSenti is released under CC BY-NC-SA 4.0, and the authors' ethics statement forbids commercial use. The data is used here for non-commercial coursework and is not redistributed in my repository; a download script fetches it instead.

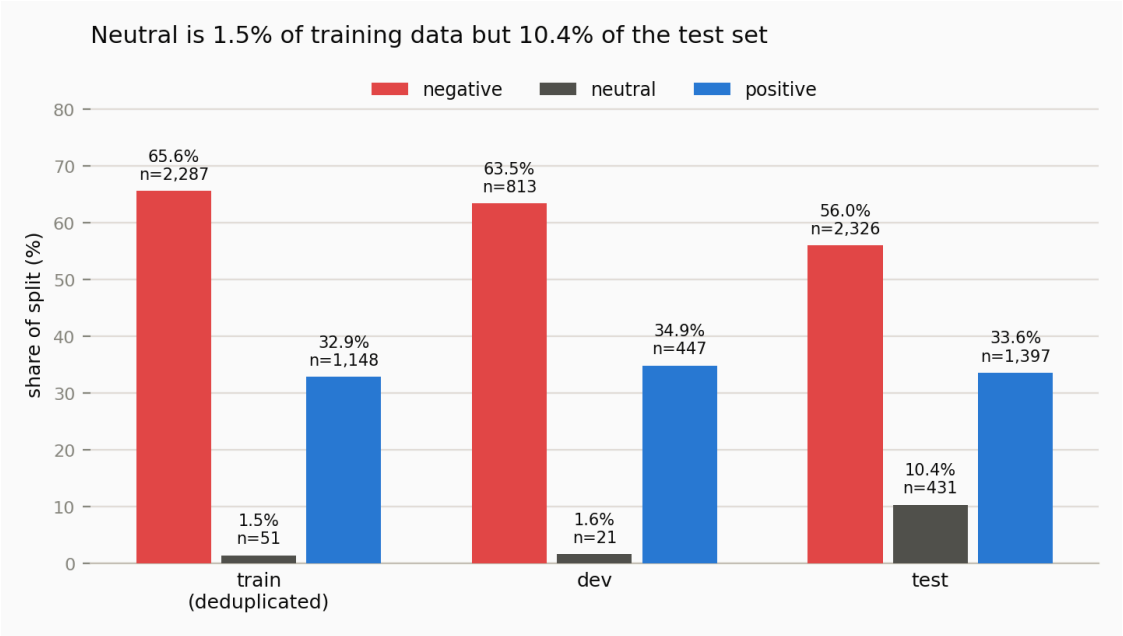
3.2 What the audit found

I wrote an audit script rather than trusting the release, and it surfaced two problems.

The neutral class is distributed differently in training and test.

	train	dev	test
negative	3,241	813	2,326
positive	1,808	447	1,397
neutral	72	21	431
neutral share	1.4%	1.6%	**10.4%**

A model trained on this sees almost no neutral examples and is then tested on a split where neutral is a tenth of the data.



Class distribution across the three splits. Neutral is 1.5% of training data and 10.4% of the test set.

Figure 1. Label distribution by split. The training and test splits are drawn from different label distributions, and the neutral class is the one that moves.

The release leaks, and in places contradicts itself.

Check	Count
duplicate tweets inside train	512
train tweets appearing verbatim in test	1,013
train tweets appearing verbatim in dev	291
dev tweets appearing verbatim in test	266
share of test reachable verbatim from train	22.3%
gold label agreement on the train/test overlap	87.8%

Just over a fifth of the test set can be found word for word in the training data. On those shared tweets the two splits disagree about the correct label 124 times, so some portion of any measured error rate is the benchmark disagreeing with itself rather than the model being wrong.

3.3 Three training files

Rather than pick one response to this, the preparation script writes three training files and leaves dev and test untouched:

- train_full, the release as distributed, whitespace-normalised only. Results on this stay comparable to published numbers, which were computed on these splits.
- train_dedup, with duplicates removed and any tweet that also appears in dev or test dropped. 3,486 rows instead of 5,121.
- train_full_sub, a size-matched control: 3,486 rows sampled from the full release, so the leakage is retained but the row count matches train_dedup.

The third file exists because train_full is 47% larger than train_dedup, so comparing them would confound the leakage with the extra data. The control holds the row count fixed and lets the leakage be measured on its own.

3.4 Preprocessing

Preprocessing is deliberately light. AfriSenti is already lowercased, stripped of punctuation, and anonymised with mentions replaced by @user and URLs removed. The only steps I apply are collapsing runs of whitespace and dropping rows that end up empty. Emojis are kept: 275 training tweets contain them, and they turn out to carry sentiment, as Section 7 shows.

3.5 The finance and crypto test set

No labelled Pidgin data exists for financial text, so I built a small test set.

Posts were collected from the Nigeria (Naija) local board on Bitcointalk, a public forum where Pidgin is explicitly permitted: a moderator policy of 28 August 2026 allows thread creators to designate a thread as Pidgin-only or English-only. I checked the site's robots.txt, which contains no restrictions, and collected with a rate limit of one request every 1.5 seconds, caching pages so the board was not fetched twice for the same page. Quoted text and signature adverts were stripped so each row is one person's own words. No usernames were recorded; the posts are used as language samples and author identity is irrelevant to the task.

Candidate posts were filtered for Pidgin markers and financial vocabulary. The first version of that filter matched substrings and was wrong: " na" matched " naira" and "im" matched "him", so standard English passed through as Pidgin. I rewrote it with word boundaries and two tiers of markers, where words that are almost never standard English count on their own and words that are ordinary English too require several. Precision on a hand-checked sample went from roughly 30% to roughly 80%. This is worth recording because the failure would have been invisible after labelling: the resulting test set would have been mostly English while appearing to be Pidgin.

A length constraint also applies. Forum posts are much longer than tweets: of 21,378 posts scanned, 94% exceeded 60 words, while the longest AfriSenti test post is 63 words. I capped collection at 60 words so that the finance set and the AfriSenti test set overlap in length, which Section 7 relies on.

Labelling followed written guidelines fixed before any post was labelled. The rules label the author's stance towards the financial subject rather than the mood of the post: "Bitcoin crash again" is neutral, while "Bitcoin crash again, I done tire" is negative. Scam warnings are negative, price reports without a stance are neutral, and posts that are not recognisably Pidgin or not about money are excluded.

	count
posts collected and screened	130
excluded under the guidelines	20 (15%)
usable test posts	**109**

Of the exclusions, nine were recorded as standard English rather than Pidgin, which is the filter's residual error rate showing up where it should: at the annotator.

The resulting label distribution is nothing like AfriSenti's. Figures below are from this first labelling; Sections 4.6 and 4.7 report a second independent pass and the adjudicated set of 93 posts that the results actually use.

class	**first labelling**	**adjudicated set**	**AfriSenti test**
negative	18 (16.5%)	16 (17.2%)	56.0%
neutral	**51 (46.8%)**	**41 (44.1%)**	10.4%
positive	40 (36.7%)	36 (38.7%)	33.6%

Nigerian crypto forum discussion is mostly informational. People explain how wallets work, how to move naira to an exchange, and what a seed phrase is. AfriSenti tweets are mostly complaints. The neutral share stays near 45% under every labelling, against 10.4% in AfriSenti.

3.6 Annotation reliability

A second native Pidgin speaker labelled 126 of the 130 posts independently, working from the same written guidelines and without seeing the first set of labels. 125 posts carry a label from both.

judgement	**raw agreement**	**Cohen's kappa**
what to exclude (not Pidgin, not finance)	85.6%	**0.539**
positive against negative, neutral dropped	69.7%	**0.368**
all three sentiment classes	40.2%	**0.088**

The three-way figure is poor. Muhammad et al. (2023b) report a free marginal kappa of 0.60 for Pidgin in AfriSenti, and 0.088 is far below it.

The decomposition shows the disagreement is concentrated on one class. The two annotators agree moderately on what to exclude and reasonably on which pole a post sits at; they do not agree on neutral. Annotator 1 assigned neutral to 50 posts and annotator 2 to 34, and annotator 1's neutrals scatter across all three of annotator 2's categories.

	annotator 2: negative	**neutral**	**positive**
annotator 1: negative	8	6	4
neutral	10	14	18
positive	6	11	15

The guidelines did not pin down where "explaining how a wallet works" stops and "recommending a wallet" begins, and that is where the two readings part. Low agreement on a category is evidence about the category as well as about the annotators: two native speakers working from written rules could not agree on what counts as neutral in Pidgin financial discussion, and the model cannot predict the class at all. Section 7.5 tests whether the headline result survives this, and it does.

3.7 Adjudication

Rather than report a test set with this level of disagreement, the two annotators worked through the 73 contested posts together in a third pass. The 52 posts they had already agreed on were left untouched: reopening settled items would turn two independent passes into one negotiated one.

All 73 were settled, none left unresolved. The final labels match annotator 1 on 73.6% of posts and annotator 2 on 60.0%, so this was not one annotator conceding to the other. The largest group, neutral against positive, split 14 to 14, which is itself informative: on the most contested question the pair decided case by case rather than converging on a rule.

The adjudicated set holds **93 usable posts**: 16 negative, 41 neutral, 36 positive, with 32 excluded.

The adjudication settled the labels but did not sharpen the guideline. Each decision carries a recorded reason, and those reasons were applied unevenly. The exclusion criteria were consistent, with "out of context" and "english" mapping to exclude every time, and "fact" to neutral in 92% of cases. But "statement" and "sentiment", covering 42 of the 73 decisions, describe what the annotators saw rather than a rule they applied, and neither maps reliably to a single label. The informational-against-approving boundary should be defined with worked examples before labelling begins; it cannot be recovered afterwards.

With 93 posts, 16 of them negative, per-class results on this set remain indicative rather than precise and are reported as such.

4 METHODOLOGY

4.1 Baseline

TF-IDF features into a multinomial logistic regression. It needs no pretraining, trains in seconds, and sets the number every later model has to beat. I compared word n-grams (1–2) against character n-grams (3–5) because Pidgin spelling is not standardised, so the same word appears as dey, de and dy. Character n-grams share evidence across those variants where word n-grams treat them as unrelated. Terms appearing in only one tweet are dropped, since on a corpus this small they are mostly typos.

4.2 Word2Vec and BiLSTM

A bag-of-words model cannot distinguish “no be scam” from “scam”. A BiLSTM reads words in order, so negation and word order can matter. The embedding layer is initialised from Word2Vec vectors trained on the project’s own training split, using skip-gram with negative sampling, because no large pretrained Pidgin Word2Vec model exists.

The implementation pads each batch to its own longest sequence and uses pack_padded_sequence, so the LSTM never reads padding as if it were words. Pooling takes both the mean and the max over all timesteps rather than the final hidden state, because these posts are short and the sentiment-carrying word can sit anywhere.

4.3 Transformers

Four pretrained encoders, chosen to separate kinds of pretraining:

model	**pretraining**	**what it tests**
roberta-base (Liu et al., 2019)	English only	how far English pretraining alone gets you on an English-lexifier creole
xlm-roberta-base	100 languages, not Pidgin	whether general multilingual pretraining helps
Davlan/afro-xlmr-base	17 African languages including Pidgin	whether targeted adaptation helps
castorini/afriberta_base	11 African languages, under 1 GB	whether targeted but small pretraining beats broad but unfocused

I deliberately excluded Davlan/naija-twitter-sentiment-afriberta-large. It is already fine-tuned on NaijaSenti, which is the same Pidgin data, so scoring it here would be testing it on its own training data.

4.4 Training

The training loop is written out rather than handed to a high-level trainer, so every choice is visible. AdamW, linear warmup over the first 10% of steps then linear decay, no weight decay on bias and LayerNorm parameters, gradient clipping at 1.0, mixed precision, batch

size 16, learning rate 2e-5, maximum length 128 tokens with per-batch padding, seed 42 unless stated.

Each batch is padded to its own longest sequence rather than to 128. The median post is 22 tokens, and the 95th percentile is 50, so padding everything to 128 would spend most of the compute on padding. On a 6 GB GPU, this was the difference between fitting and thrashing.

The epoch budget is six, with the best epoch selected on dev macro F1. I originally used three and had to change it. At three epochs, AfroXLM-R had already peaked at epoch two and was declining, while XLM-R was still improving. A budget that one model has outgrown and another has not yet reached is not a fair comparison. At the corrected budget, the apparent gap between them fell from 5.1 macro F1 points to 1.2.

4.5 Class weighting

Each class is weighted by inverse frequency, $w_c = N / (K \cdot n_c)$, where N is the number of training rows, K the number of classes, and n_c the count of class c. With 51 neutral rows, this gives neutral a weight of about 22.8 against 0.51 for negative.

These weights are not rescaled afterwards. Their expected value under the training distribution is already exactly 1:

$$\Sigma_c (n_c/N) \cdot N/(K \cdot n_c) = \Sigma_c 1/K = 1$$

An earlier version divided by the arithmetic mean of the weights instead, which dragged the common classes down to around 0.06 and shrank the whole loss to roughly a tenth of its unweighted size, quietly reducing the effective learning rate. The derivation above is why the raw weights are correct as they stand.

4.6 Evaluation

Both macro and weighted F1 are reported throughout, along with accuracy. Weighted F1 is the metric the SemEval-2023 leaderboard used, so it is what published numbers compare against. Macro F1 treats the three classes equally, which is what exposes the neutral failure. Section 6 shows that the two disagree about which model is better, and why.

Model selection uses dev macro F1. This introduces a known bias: the best epoch is chosen on dev, and that epoch's dev score is reported, so dev numbers are optimistic by construction. Every model receives the same treatment, so comparisons between them are fair, but the clean generalisation estimate is the test split, which was scored once at the end.

5 EXPERIMENTS AND RESULTS

5.1 Model comparison

All models at the six-epoch budget, scored on dev.

model	pretraining	accuracy	macro F1	weighted F1	F1 neutral
TF-IDF + logistic regression	none	0.665	0.424	0.658	0.000
Word2Vec + BiLSTM	none	0.671	0.422	0.659	0.000
AfriBERTa	African only, <1 GB	0.687	0.437	0.677	0.000
XLM-R	100 languages, no pcm	0.712	0.463	0.707	0.000
RoBERTa	English only	0.728	0.467	0.718	0.000
AfroXLM-R	17 African langs, incl. pcm	**0.743**	0.471	0.727	0.000

Three things come out of this table.

English-only pretraining beats general multilingual pretraining. RoBERTa has never seen an African language and still beats XLM-R, which covers 100 languages but not Pidgin. This makes sense for an English-lexifier creole: most Pidgin vocabulary is English-derived, so an English subword vocabulary segments it well, while XLM-R spends its capacity on languages that do not help here. This answers the first research question: targeted adaptation wins, but the runner-up is English rather than multilingual coverage.

The ordering matches the published one. Muhammad et al. (2023b) report AfriBERTa < XLM-R < AfroXLMR on this language, and the same ordering holds here. Their absolute values are lower because they are test numbers and these are dev, and the test split contains six times as much neutral data.

The sequence model does not beat bag-of-words. The BiLSTM and TF-IDF are level at 0.671 against 0.665 accuracy, inside the noise floor established in Section 6.4. The brief notes that a more complex model need not perform better, and here it does not. The training curve shows why: training loss fell from 1.12 to 0.39 over 12 epochs while dev macro F1 plateaued around 0.41 from epoch 6. That is 610,559 parameters memorising 3,486 examples. The Word2Vec vectors are also weak, having been trained on roughly 85,000 tokens where Word2Vec normally expects millions; an ablation with random embeddings scores 0.618, so the vectors are worth about 2.6 accuracy points but no more.

5.2 Class weighting

model	Effect of inverse-frequency weighting
TF-IDF	+5.6 macro F1
Word2Vec + BiLSTM	−2.8 accuracy, macro F1 unchanged
AfroXLM-R	no reliable effect; the sign flips between budgets

One technique, three different outcomes. The explanation is that weighting moves the decision boundary towards rare classes but supplies no new information. TF-IDF was collapsing onto the majority class, so moving the boundary rescued positive recall from 0.22 to 0.51 and bought a lot. The BiLSTM and the transformer were not collapsing in that way, so the same move only cost them accuracy on classes they already handled. None of them recovers neutral, because 51 examples is an information problem and weighting is not information.

5.3 How much the leakage inflates scores

Same model, same budget, same seed. Only the training file differs.

training file	rows	leakage	accuracy	macro F1	F1 neutral
train_dedup	3,486	none	0.743	0.472	0.000
train_full_sub	3,486	retained	0.748	0.511	0.077
train_full	5,121	retained	0.756	0.521	0.083

The middle row is the control. With the row count held fixed, leakage alone is worth about 4 macro F1 points; the additional 1,635 rows add roughly 1 more. So the inflation decomposes to roughly 80% leakage and 20% extra data.

The neutral column is the clearest sign of what is happening. Neutral only becomes non-zero in the leaky conditions. The model is not learning the class; it is recognising tweets it has already seen.

5.4 Measurement noise

Two different sources of variation, which should not be conflated.

source	size (macro F1)	how measured
seed to seed	0.008	three seeds, same configuration, test split
epoch budget	0.016	same seed, three epochs against six, dev split

Changing the budget changes the entire learning-rate schedule, because the linear decay is computed over total training steps. Epoch two of a three-epoch run and epoch two of a six-epoch run sit at different points on that curve. It moves the result about twice as much as reseeding does. Any difference between models smaller than these figures is not a difference.

5.5 Final result

AfroXLM-R, class-weighted, trained on train_dedup, six epochs, best epoch chosen on dev. Test scored once, across three seeds.

metric	mean	range
accuracy	**0.697**	0.694 – 0.700

metric	mean	range
macro F1	0.479	0.475 – 0.483
weighted F1	0.658	0.654 – 0.663
F1 negative	0.784	0.781 – 0.785
F1 neutral	**0.000**	0.000 – 0.000
F1 positive	0.653	0.644 – 0.665

Muhammad et al. (2023b) report 67.6 accuracy for AfroXLMR-base on this split, averaged over five runs. This project reaches 69.7 averaged over three runs while training on 3,486 rows rather than 5,121. Removing a third of the training data cost nothing measurable, because the third removed was the leaked third. The comparison is across different numbers of runs, three against five, and should be read as agreement in direction and rough magnitude rather than as a replication.

The neutral row is the result that matters. An F1 of exactly 0.000 on all three seeds, with zero spread, against 431 neutral test examples. The model never predicts the class.

5.6 Can more neutral data rescue it?

The obvious response is that the model needs more neutral examples. I tested that.

NaijaSenti adds nothing, since its Pidgin files are identical to AfriSenti's. Two smaller corpora do add examples, both CC BY 4.0: WAZOBIALABS' nigerian-pidgin-voice-text (500 rows, 77 neutral) and nigerian-pidgin-eval (230 rows, 11 neutral). Together they contribute 67 usable neutral rows after removing duplicates and overlap, taking the training split from 51 neutral to 118.

training split	neutral rows	neutral weight	F1 neutral	neutral recall	accuracy
train_dedup	51	22.78	0.000	0.000	0.697
train_augmented	118	11.14	**0.035**	0.019	0.696

The model predicts neutral for the first time: 8 of the 431 neutral test posts, at 0.286 precision.

The weight column is the part to read twice. Adding neutral examples also lowers their inverse-frequency weight, from 22.78 to 11.14. The augmented run therefore had half the loss pressure towards neutral and still did better. Sixty-seven extra examples beat halving the weight, which separates evidence from reweighting about as cleanly as this data allows.

It remains a broken classifier. A recall of 1.9% is not usable, and overall accuracy is unchanged within seed noise. Doubling the neutral evidence bought almost nothing in practice, which suggests the shortfall runs to thousands of examples rather than dozens.

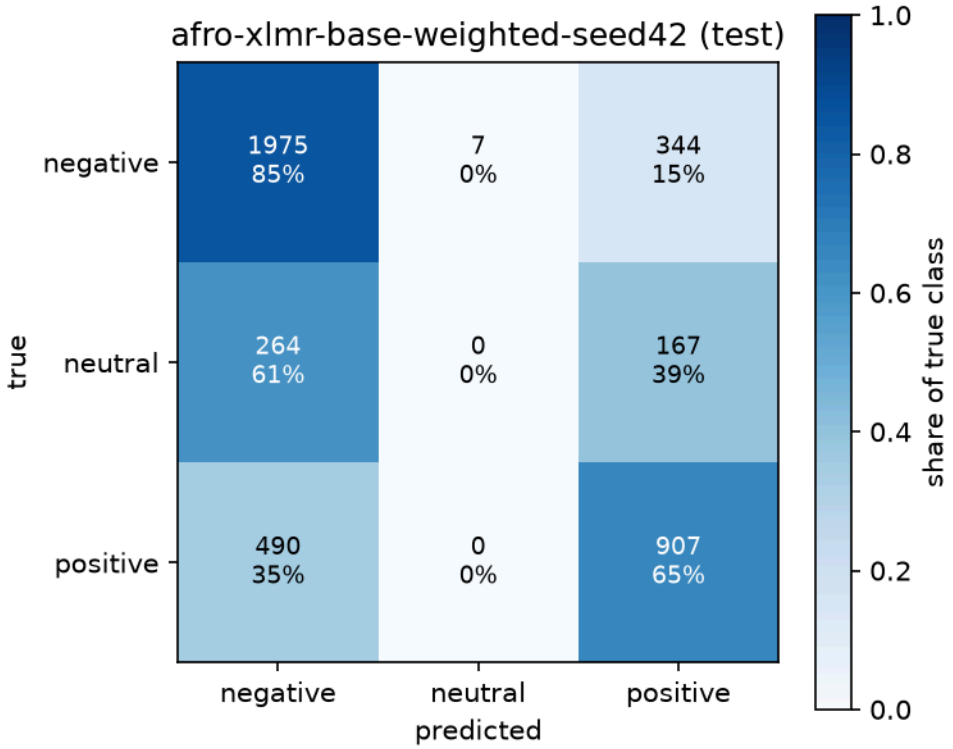
Two caveats: the added rows are transcribed speech rather than tweets, so the register differs; and nigerian-pidgin-eval is distributed as an evaluation set, so it is used here only to test the scarcity hypothesis and never to report a score.

6 ERROR ANALYSIS AND DISCUSSION

6.1 Where the errors are

Test predictions from the seed-42 run: 4,154 posts, 1,272 wrong.

true → predicted	count	share of that true class
negative → negative	1,975	84.9%
negative → positive	344	14.8%
neutral → negative	264	**61.3%**
neutral → positive	167	**38.7%**
positive → negative	490	35.1%
positive → positive	907	64.9%



Confusion matrix on the AfriSenti test split. The neutral row is empty: no neutral prediction is ever made.

Figure 2. Confusion matrix on the AfriSenti test split, seed 42. Rows are true labels, columns are predictions, shaded by row share. The neutral column is empty.

Neutral does not fail at random. It splits between the two poles, leaning negative, which is the direction the training distribution leans. The model has no neutral region at all; every post is pushed to one side or the other.

6.2 Length

post length	posts	error rate
1–10 words	1,189	**0.405**
11–20 words	1,503	0.310
21–35 words	995	0.229
36+ words	467	**0.206**

Short posts are twice as hard as long ones. There is simply less evidence in them, and with a language whose spelling varies, a short post may contain only one or two informative tokens.

6.3 Two results that contradict the obvious explanation

The intuitive story for a low-resource creole is that unseen vocabulary drives the errors, because spelling is unstandardised. The data does not support it.

share of words unseen in training	posts	error rate
0 – 4.8%	2,093	0.312
4.8 – 10.5%	1,027	0.273
10.5 – 66.7%	1,034	0.329

The error rate barely moves across the range and is not monotonic. Whatever is going wrong, it is not primarily out-of-vocabulary words.

Emoji run the other way from expectation too. Posts containing emoji have an error rate of 0.244 against 0.310 for posts without. Emoji carry explicit sentiment that the text sometimes does not, and the subword tokeniser keeps them.

6.4 Errors that are not the model's fault

Of the test posts, 926 also appear in the training split, and 113 of those carry a different label in the two places. In 24 cases, the model predicted the label the training split taught it and was marked wrong by the test split. Those are not modelling failures; they are a ceiling on the achievable score imposed by the benchmark contradicting itself.

6.5 Domain transfer

This is the part of the project that is not a replication.

The finance posts are longer than AfriSenti's tweets, so the comparison is made against a length-matched subset of the AfriSenti test set: the 2,146 posts that fall within the finance

set's own range of 16 to 64 words. Without that control, a difference in score could be explained by post length rather than by domain.

metric	AfriSenti, length-matched	finance test	change
accuracy	0.759	**0.355**	−0.404
macro F1	0.503	0.304	−0.199

An accuracy of 0.355 on three classes is barely above chance. But the headline number confounds two effects, and separating them is the analysis.

component	size	what it is
blind spot meeting the domain	24.6 points	Neutral is 44.1% of the finance set against 4.3% of the matched sample, and the model never predicts neutral, so its ceiling here is 52/93 = 55.9% before it classifies anything
genuine domain difficulty	15.8 points	With neutral removed from both sides, accuracy falls from 0.793 to 0.635

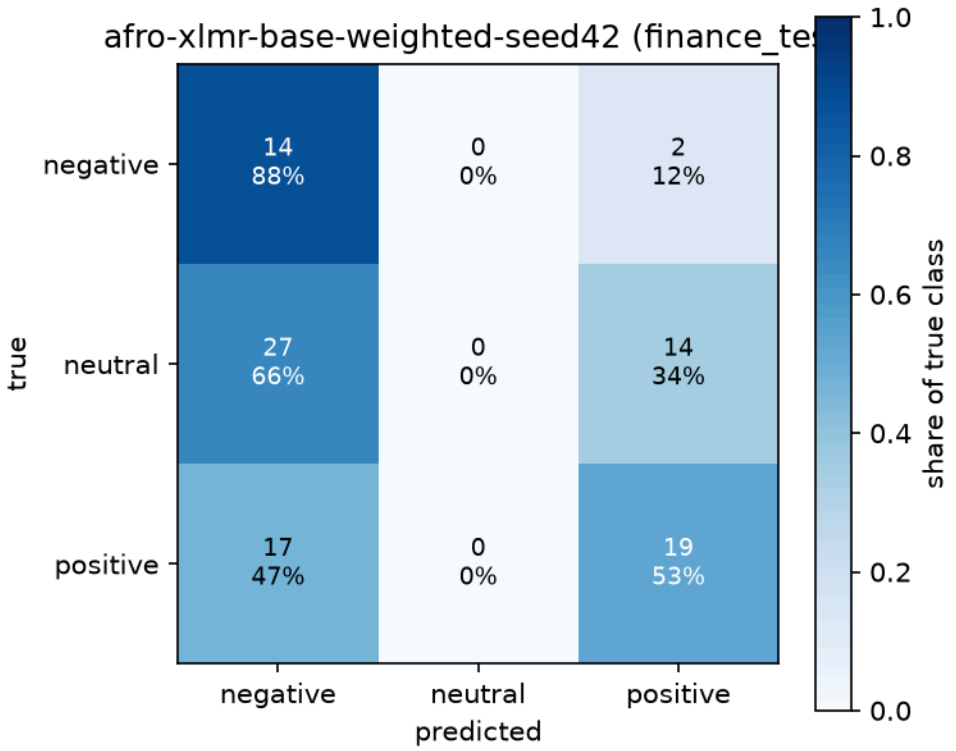
Both are real, and they say different things. The first is a property of the training data meeting a different world. The second is the model being genuinely worse at reading Pidgin about money, on the two classes it can handle at all.

Does this survive the low annotator agreement? Section 4.6 reports a three-way kappa of 0.088, so the labels behind these numbers are not reliable. The result was therefore recomputed against each annotator separately, and against the subset where the two independently agreed.

labels	n	neutral share	accuracy	macro F1
annotator 1 alone	105	47.6%	0.333	0.294
annotator 2 alone	92	33.7%	0.413	0.332
independently unanimous	37	37.8%	0.514	0.415
adjudicated (reported)	**93**	**44.1%**	**0.355**	**0.304**
AfriSenti, length-matched	*2,146*	*4.3%*	*0.759*	*0.503*

The model collapses under every labelling. Even on the 37 posts the two annotators independently agreed on, accuracy is 0.514 against the control's 0.759. The neutral share is between 34% and 48% for whoever labelled, against 10.4% in the AfriSenti test split.

The adjudicated set is the one reported above, and it sits almost exactly where the first annotator's set did: 0.355 against 0.333 accuracy, 0.304 against 0.294 macro F1, neutral F1 of 0.000 either way. The finding did not depend on which annotator was right about the contested posts, which is the strongest thing that can be said for it given a pre-adjudication kappa of 0.088.



Confusion matrix on the finance test set. Nearly half the posts are neutral and none are recognised.

Figure 3. Confusion matrix on the adjudicated finance test set. 41 of the 93 posts are neutral and the model assigns none of them correctly, which caps its accuracy at 55.9% before it classifies anything.

The mechanism. On the matched AfriSenti sample the model predicts negative 66.5% of the time, and 61.7% really are negative. On the finance set it still predicts negative about 62% of the time whether the true rate is 16.5% under the first labelling or 26.1% under the second.

It has carried its training prior across unchanged. What it learned is closer to "most Pidgin tweets are negative" than to "this is what negative Pidgin looks like". Nigerian crypto discussion is mostly people explaining how wallets and exchanges work, so that prior is wrong, and the model applies it anyway.

6.6 Relation to the literature

Muhammad et al. (2023b) report sarcasm and code-mixing as sources of annotator disagreement across several AfriSenti languages. I found the same thing while labelling: sarcastic posts where the surface words are positive and the meaning is not were the hardest category to label consistently, and the model fails on them too. A constructed probe, "na very good exchange, dem swallow my money since january", is classified positive with 0.91 confidence.

The confidence figures deserve a note of their own. The model is between 0.69 and 0.92 confident on the examples it gets wrong, and 0.68 confident on one it gets right. Softmax

confidence does not separate its correct answers from its incorrect ones, so there is no threshold at which a deployed system could sensibly abstain.

7 DEPLOYMENT

The model is published on the Hugging Face Hub at HonourGod001/pidgin-sentiment-afroxlmr, and the interface is a Streamlit application that loads it from there, so the same file runs locally and in the cloud without modification, and no checkpoint is committed to the repository.

The interface is a chat: the user writes Pidgin, the model replies with the predicted label, its confidence, and a bar for each of the three classes. The connection to the model is direct — the app tokenises the input, runs a forward pass, applies a softmax over the three logits, and reads the label order from the model configuration rather than assuming it.

Hugging Face Spaces was the first choice but now requires a paid plan for Gradio applications, so the app is hosted on Streamlit Community Cloud instead.

The interface states its own limits rather than hiding them. A sidebar panel explains that the model has exactly three outputs and no way to abstain, and that neutral is effectively never predicted. Under each reply, the app warns when the input shows no sign of being Pidgin and when confidence is low. Four example inputs are offered, of which two are cases the model gets wrong.

8 LIMITATIONS AND FUTURE WORK

The neutral class. The headline limitation. 51 training examples against roughly 3,400. Adding every other labelled neutral Pidgin corpus I could find took it to 118 and lifted F1 from 0.000 to 0.035, so the shortfall runs to thousands of examples. Collecting and annotating neutral Pidgin at scale is the single most useful thing anyone could do for this task.

The finance test set is small, and its labels were initially unreliable. 93 posts after adjudication, 16 of them negative. Two native speakers labelled independently, and three-way agreement was Cohen's kappa of 0.088, far below the 0.60 reported for Pidgin in AfriSenti, with the disagreement concentrated on the neutral class. The 73 disagreements were then adjudicated jointly, and the reported result uses that adjudicated set. It matches the pre-adjudication result closely, so the finding does not rest on the contested posts, but a test set this small whose first pass agreed this poorly cannot support precise per-class claims.

The fix is not more annotators but a tighter definition. The guidelines did not say where informational Pidgin stops and approving Pidgin begins, and that is precisely the boundary the two annotators drew differently. A future version should define the neutral category with worked examples of that boundary, label in two passes, and adjudicate disagreements rather than reporting a single pass.

The platform is not controlled. Post length is matched between the finance set and its AfriSenti control, but the finance posts are forum replies and the AfriSenti posts are tweets. Some unknown part of the 15.5-point domain effect is register rather than subject matter. Separating the two would need Pidgin financial text from Twitter, which does not exist in labelled form.

Three seeds, not five. The published baselines average over five runs. Three was what the time budget allowed.

Hardware. A 6 GB laptop GPU restricted the work to base-size models. AfroXLMR-large scores 68.7 on this split in the published table, so a larger model is likely to do better.

No calibration work. The confidence scores are raw softmax outputs and are demonstrably unreliable, which matters more for deployment than the headline accuracy does.

Future work follows from the limitations. The most valuable next step is a labelled Pidgin corpus with a realistic neutral share, which the finance set begins in a small way: its 51 neutral posts are a meaningful addition relative to the 51 in the existing training split. After that, a Pidgin-specific adaptation of AfroXLM-R on unlabelled Pidgin text, which does exist in quantity, would be worth testing.

9 CONCLUSION

A fine-tuned AfroXLM-R reaches 0.697 accuracy on the AfriSenti Pidgin test split, slightly ahead of the published figure for the same model, while training on a deduplicated split that is a third smaller. On the benchmark, it looks like a competitive model.

It is not a usable one. It never predicts neutral, on any seed, across 431 opportunities. On Pidgin financial text, which is 44% neutral, it scores 0.355 against a length-matched control's 0.759 — barely above chance on three classes. It does this while carrying its training prior unchanged into a domain where that prior is wrong.

The three things I would take from the project are these. First, audit the benchmark before trusting it: this one leaks 22% of its test set into training and contradicts itself on 124 tweets, and the leakage alone is worth 4 macro F1 points. Second, a benchmark score and a usable system are different claims, and the gap between them can be the width of a domain. Third, when a class fails, check whether the cause is method or data before trying to fix it: here, a 22× loss weight changed nothing and 67 extra examples changed something, which settles the question.

REFERENCES

Alabi, J. O., Adelani, D. I., Mosbach, M., & Klakow, D. (2022). Adapting pre-trained language models to African languages via multilingual adaptive fine-tuning. In *Proceedings of the 29th International Conference on Computational Linguistics* (pp. 4336–4349). International Committee on Computational Linguistics.

Adelani, D. I., Doğruöz, A. S., Coneglian, A., & Ojha, A. K. (2024). Does generative AI speak Nigerian Pidgin? Issues about representativeness and bias for multilingualism in LLMs. *arXiv:2404.19442*.

Chainalysis. (2026). *The 2026 Global Crypto Adoption Index*. Chainalysis.

Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 8440–8451). Association for Computational Linguistics.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv:1907.11692.*

Muhammad, S. H., Adelani, D. I., Ruder, S., Ahmad, I. S., Abdulmumin, I., Bello, B. S., Choudhury, M., Emezue, C. C., Abdullahi, S. S., Aremu, A., Jorge, A., & Brazdil, P. (2022). NaijaSenti: A Nigerian Twitter sentiment corpus for multilingual sentiment analysis. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference* (pp. 590–602). European Language Resources Association.

Muhammad, S. H., Abdulmumin, I., Yimam, S. M., Adelani, D. I., Ahmad, I. S., Ousidhoum, N., Ayele, A. A., Mohammad, S., Beloucif, M., & Ruder, S. (2023a). SemEval-2023 Task 12: Sentiment analysis for African languages (AfriSenti-SemEval). In *Proceedings of the 17th International Workshop on Semantic Evaluation* (pp. 2319–2337). Association for Computational Linguistics.

Muhammad, S. H., Abdulmumin, I., Ayele, A. A., Ousidhoum, N., Adelani, D. I., Yimam, S. M., Ahmad, I. S., Beloucif, M., Mohammad, S. M., Ruder, S., Hourrane, O., Brazdil, P., Jorge, A., Ali, F. D. M. A., David, D., Osei, S., Bello, B. S., Ibrahim, F., Gwadabe, T., … Arthur, S. (2023b). AfriSenti: A Twitter sentiment analysis benchmark for African languages. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 13968–13981). Association for Computational Linguistics.

Ogueji, K., Zhu, Y., & Lin, J. (2021). Small data? No problem! Exploring the viability of pretrained multilingual language models for low-resourced languages. In *Proceedings of the 1st Workshop on Multilingual Representation Learning* (pp. 116–126). Association for Computational Linguistics.

Datasets and models

AfriSenti (Nigerian Pidgin split), CC BY-NC-SA 4.0. https://github.com/afrisenti-semeval/afrisent-semeval-2023

Davlan/afro-xlmr-base, MIT. https://huggingface.co/Davlan/afro-xlmr-base

castorini/afriberta_base. https://huggingface.co/castorini/afriberta_base

WAZOBIALABS Nigerian Pidgin datasets, CC BY 4.0. https://huggingface.co/datasets/WAZOBIALABS/nigerian-pidgin-voice-text

Libraries

PyTorch, Hugging Face Transformers, scikit-learn, Gensim, pandas, Matplotlib, Streamlit.

Acknowledgement of AI assistance

I used an AI coding assistant while building this project to write and debug code, draft documentation, and discuss analysis. I verified the results myself: every number in this report traces to an output file in results/ produced by a script in src/, and the commands that produced them are listed in reproduce.sh. I am responsible for the design decisions, annotations, and interpretations presented here.